

SUPPLEMENTARY PROBLEM: MIN-MAX OF AN ARRAY

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Problem Statement

Design a divide-and-conquer algorithm to find both the **minimum** and the **maximum** element of n numbers using no more than $3n/2$ comparisons.

Intuition

Split the problem into 2 parts, A and B, by taking the first and last index and dividing by 2. Note that this would divide unevenly if there are an odd number of elements. To minimize the number of comparisons, do 1 comparison when there are 2 elements in the subproblem and 0 comparisons when there is only 1 element in the subproblem. Then, return both the min and max of that section, and doing 2 comparisons to merge two chunks: one between the two minima and one between the two maxima. This will find the overall min and max of both parts. Note that the algorithm could use the min and max functions to find the min and max, but in order to further elucidate the number of comparisons, if conditions that compare the 2 mins and maxes are used instead.

Pseudocode

Algorithm 1 Minmax Algorithm

Require: Array of comparable elements, A

Require: Index of first element, f (will most likely be 0)

Require: Index of last element, l (will most likely be $n - 1$, where n is the size of the array)

```
1: procedure FIND_MIN_MAX( $A, f, l$ )
2:    $\text{length} \leftarrow l - f + 1$ 
3:   if  $\text{length} == 1$  then return  $A[f], A[l]$ 
4:   else if  $\text{length} == 2$  then
5:     if  $A[f] > A[l]$  then
6:       return  $A[l], A[f]$ 
7:     else
8:       return  $A[f], A[l]$ 
9:     end if
10:  else
11:     $\text{mid} \leftarrow \lfloor (f + l) / 2 \rfloor$ 
12:     $\text{minA}, \text{maxA} \leftarrow \text{FIND\_MIN\_MAX}(A, f, \text{mid})$ 
13:     $\text{minB}, \text{maxB} \leftarrow \text{FIND\_MIN\_MAX}(A, \text{mid} + 1, l)$ 
14:     $\text{min} \leftarrow +\infty$ 
15:     $\text{max} \leftarrow -\infty$ 
16:    if  $\text{minA} < \text{minB}$  then
17:       $\text{min} \leftarrow \text{minA}$ 
18:    else
19:       $\text{min} \leftarrow \text{minB}$ 
20:    end if
21:    if  $\text{maxA} > \text{maxB}$  then
22:       $\text{max} \leftarrow \text{maxA}$ 
23:    else
24:       $\text{max} \leftarrow \text{maxB}$ 
25:    end if
26:    return  $\text{min}, \text{max}$ 
27:  end if
28: end procedure
```

Proof of Correctness

Proof. This proof will employ **strong induction** to show that the algorithm correctly identifies the minimum and maximum values for arrays of any size:

Begin with the **base cases**.

1. Consider a subarray of **1 element**. In this case, the single element is both the minimum and maximum of the subarray.
2. Consider a subarray of **2 elements**. In this case, if the first element is larger than the second element, then the first element is the maximum and the second element; and vice versa.

This demonstrates that the algorithm can locate the minimum and maximum elements for arrays of size 1 or 2.

Next, consider the **induction hypothesis**. In this case, assume that the algorithm correctly returns the minimum and maximum element for any array with a size $< k$, for some positive k .

Inductive Step: When the algorithm splits an array L of size k into two parts, A and B, each part must have a size of at most $\lceil \frac{k}{2} \rceil$, which is $< k$. By the induction hypothesis, the algorithm correctly returns the minimum and the maximum element for both A and B:

- A_{min} : minimum of A
- B_{min} : minimum of B
- A_{max} : maximum of A
- B_{max} : maximum of B

Considering the minimums:

The minimum of the entire list L would then simply be $\min(A_{min}, B_{min})$. If the $A_{min} < B_{min}$, then the minimum of A is returned as the minimum of L; conversely, if $A_{min} > B_{min}$, then the minimum of B is returned as the minimum of the entire list. If $A_{min} = B_{min}$, then the algorithm returns the minimum of B; in this case, either elements would be considered the minimum element as both are identical. Thus, the algorithm returns the correct minimum element for an array of size k .

The same argument applies to the maximum values.

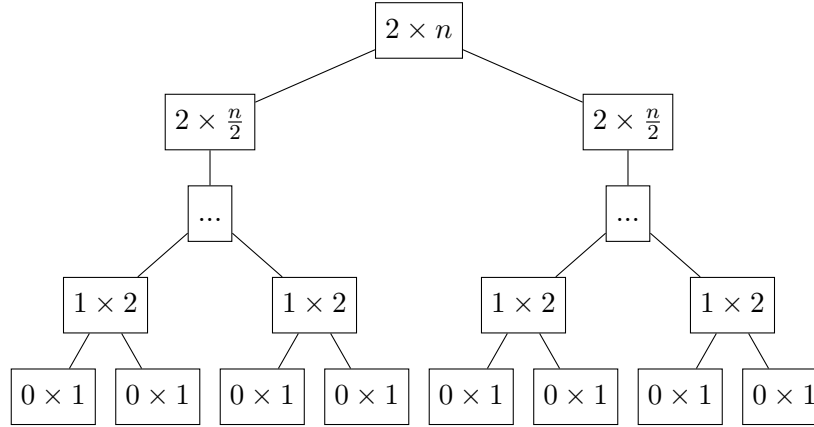
Thus, it is shown by induction that the algorithm can correctly return the minimum and maximum values for arrays of any arbitrary size.

□

Run Time - Number of Comparisons

The number of comparisons done by the algorithm is $\frac{3n}{2} - 2$, which is less than $\frac{3n}{2}$.

Proof. Below is the algorithm's recursion tree, in the worst-case scenario where every recursion ends with a single element. Each node is labeled with a value $x \times y$, where x is the number of comparisons at each call, and y is the size of the list the node is operating on.



There are 3 things to note.

1. Besides the leaf nodes and their immediate parents, each cell performs 2 comparisons: one for the minimum elements and one for the maximum elements.
2. The leaf nodes perform no comparisons: this is because there is only one element in each, and it is trivial to determine the maximum and minimum value in that case.
3. The immediate parents of the leaf nodes perform only 1 comparison, between the two leaf values. This is because there are only 2 elements, so only one comparison is needed to determine the maximum and minimum.

If the original array contains n elements, then the first level of the tree would have n elements, the second level would have $\frac{n}{2}$ elements, and the x th level would have $\frac{n}{2^{x-1}}$ elements. The height of the tree h , is then given by the equation $\frac{n}{2^h} = 1$. Solving, this provides an upper bound of $\log_2 n$ levels. Keeping in mind that the last $(\log_2 n)$ th level performs no comparisons and the second to last $(\log_2 n - 1)$ th level performs 1 comparison for $\frac{n}{2}$ leaves, the number of comparisons $C(n)$ can

be expressed as:

$$\begin{aligned}
C(n) &= 2(1 + 2 + 4 + \dots + 2^{\log_2 n - 2}) + \frac{n}{2} \times 1 + n \times 0 \\
&= 2 \sum_{i=0}^{\log_2 n - 2} (2^i) + \frac{n}{2} + 0 \\
&= 2 \frac{2^{\log_2 n - 1} - 1}{2 - 1} + \frac{n}{2} \text{ since } \sum_{i=0}^{n-1} r^i = \frac{r^n - 1}{r - 1} \\
&= 2 \frac{2^{\log_2 n} / 2^1 - 1}{1} + \frac{n}{2} \\
&= 2(n/2 - 1) + \frac{n}{2} \\
&= n - 2 + \frac{n}{2} \\
&= \frac{3n}{2} - 2 < \frac{3n}{2}.
\end{aligned}$$

Thus, the number of comparisons is less than $\frac{3n}{2}$.

Thus, the overall runtime of the algorithm is equivalent to the number of comparisons $\frac{3n}{2} - 2$:

$O(n)$

□